

Blatt 7

Kryptologie

Luis Staudt

Toy-Blockchain (SHA-256, Proof of Work)

Aufgabenstellung

Aufzubauen ist eine SHA-256-Blockchain der Länge 3, in der Rollen von Kunde und Miner zusammenfallen. Jeder Block enthält Transaktionsdaten, den Hash des Vorblocks (IV) und eine Nonce. Der *Proof of Work* verlangt, dass die führenden k Nibbles *aller drei* Block-Hashes 0 sind; dafür gibt es k Punkte.

- a) IV auf ALLZERO, Transaktionsnummer auf die Matrikelnummer setzen.
- b) Transaktionsdatenblock erzeugen (Felder (1)–(6)).
- c) Nonce so bestimmen, dass die führenden k Nibbles des Hashes 0 sind.
- d) Block um IV und Nonce ergänzen, Transaktionsnummer erhöhen, $IV := \text{Hash}$; zurück zu b).
- e) Blockchain in einem CrypTool2-Workspace nachstellen.

Ergebnis: $k = 10$ führende Null-Nibbles in allen drei Blöcken.

Verwendete Werkzeuge

- **Eigener CUDA-Miner** (`miner.cu`) auf einer NVIDIA-GPU (entwickelt/getestet auf RTX 4070, Lauf auch auf RTX 4090) für das eigentliche Mining.
- **C-Referenz** (`reference.c`) mit identischer Nachrichten-Konstruktion zur unabhängigen Prüfung auf der CPU.
- **Python** (`hashlib`) als dritte, unabhängige SHA-256-Kontrolle.
- **CrypTool2** zum Nachbau der Blockchain (`Blockchain.cwm`).

Aufbau eines Blocks

Gehasht wird die ASCII-Zeichenkette

$$\text{MESSAGE} = \text{IV} \parallel \text{TX} \parallel (8) \parallel \text{NONCE}$$

mit den Bestandteilen:

- **IV** (64 Hex): Block 1 = ALLZERO (64×0); danach jeweils der SHA-256-Hash des Vorblocks. Dies verkettet die Blöcke: $\text{IV}_n = H(\text{Block}_{n-1})$.
- **TX** (Transaktionsdaten):
 (1) Transaktionsnr (hex) (2) SHA1(Name) (3) Unixzeit (4) SHA1(Empfänger) (5) Betrag
 (6) Länge der Felder (1)–(5) (7).
- **NONCE** (32 Hex, Feld (8)): der durch Mining gefundene Wert.

Die Transaktionsnummer startet bei der Matrikelnummer ($272056 = 0x426B8$) und wird je Block um 1 erhöht. Empfänger der drei Blöcke sind Alice, Bob und Carol. Verwendete SHA1-Hashwerte:

```
1 Name "Luis Noah Staudt" = D92AB98BB95D0DA30A4C9B38CBEC5F2642AA73F9
2 Alice                   = 35318264C9A98FAF79965C270AC80C5606774DF1
3 Bob                     = DA6645F6E22BF5F75974DC7EED5FCD6160D6B51E
4 Carol                   = 6F90F101115140727C43CADEE0B9E17881403A63
```

Code Listing 1: SHA1 von Absender- und Empfängernamen

Mining (Proof of Work)

SHA-256 ist eine kryptographische Hash-Funktion ohne Abkürzung: einen Hash mit k führenden Null-Nibbles findet man nur durch Ausprobieren. Da jede Nibble mit Wahrscheinlichkeit $\frac{1}{16}$ null ist, trifft die Bedingung im Mittel jeden 16^k -ten Versuch – der Treffer folgt einer geometrischen Verteilung mit Erwartungswert 16^k Hashes je Block. Jede weitere Nibble kostet also den **16-fachen** Aufwand; das ist die Sicherheit des Proof of Work. Für $k = 10$ sind das $16^{10} \approx 1,1 \cdot 10^{12}$ Hashes je Block.

Optimierung: Midstate

Effizienz statt Trick. Statt langsam in CrypTool2 zu minen, läuft die Suche offline auf der GPU. SHA-256 verarbeitet die Nachricht in 512-Bit-Blöcken (je 64 Byte); pro Block läuft eine Kompressionsfunktion. Beim Mining ändert sich von Versuch zu Versuch nur die

Nonce ganz am Ende – der gesamte Anfang (IV + Transaktionsdaten) bleibt konstant. Diese konstanten Blöcke werden daher *einmalig* auf der CPU vorgehasht (*Midstate*); die GPU lädt den Midstate, setzt nur den variablen Zähler ein und rechnet pro Versuch lediglich die *restlichen* Kompressionsschritte.

Konkret für Block 1 (analog für die anderen):

- Nachricht = IV (64) + TX (121) + **(8)** + Nonce (32) = 220 Byte. Mit SHA-Padding ergibt das $\lceil (220 + 9)/64 \rceil = 4$ Kompressionsblöcke.
- Konstanter Präfix (IV, TX, **(8)** und die 16 zufälligen High-Hexstellen der Nonce) = 204 Byte = **3 volle Blöcke**, die als Midstate vorgehasht werden.
- Die GPU variiert nur die 16 niederwertigen Hexstellen der Nonce und rechnet je Versuch **nur den letzten** der vier Blöcke – ein Viertel der Arbeit pro Hash.

So erreicht die RTX 4070 einen Durchsatz von ≈ 3 GH/s. Bei $1,1 \cdot 10^{12}$ Hashes je Block liegt die *erwartete* Suchzeit bei ≈ 6 Minuten; wegen der geometrischen Verteilung streuen die tatsächlichen Zeiten aber stark (Standardabweichung $\approx$ Erwartungswert):

Block	Empfänger	Suchzeit
1	Alice	1260 s (≈ 21 min)
2	Bob	236 s (≈ 4 min)
3	Carol	1091 s (≈ 18 min)

Tabelle 1: Gemessene Mining-Zeiten je Block bei $k = 10$ (RTX 4090). Die große Streuung von 4 bis 21 Minuten entspricht der erwarteten geometrischen Verteilung.

Damit liegt das erreichte $k = 10$ deutlich über dem Musterbeispiel ($k = 6$, $\approx 1,7 \cdot 10^7$ Hashes).

Ergebnis: Die drei Blöcke

Gemeinsamer Zeitstempel (Feld (3)): Unixzeit **1781470311**. Jeder Hash wurde unabhängig per `reference.c`, Python `hashlib` und `CrypTool2` bestätigt; die Verkettung $IV_n = H(\text{Block}_{n-1})$ ist erfüllt.

```

1 IV   = 0000000000000000000000000000000000000000000000000000000000000000
2 TX   = (1)426B8(2)D92AB98BB95D0DA30A4C9B38CBEC5F2642AA73F9(3)1781470311
3      (4)35318264C9A98FAF79965C270AC80C5606774DF1(5)42(6)112(7)
4 NONCE= (8)591b5f1056840691000000bc249430e6

```

```
5 HASH = 0000000000161F40BEAF0B2600EE1DD39E7B34F995702AD26F88E75A6053DE94
```

Code Listing 2: Block 1 – an Alice (H mit 10 führenden Nullen)

```
1 IV = 0000000000161F40BEAF0B2600EE1DD39E7B34F995702AD26F88E75A6053DE94
2 TX = (1)426B9(2)D92AB98BB95D0DA30A4C9B38CBEC5F2642AA73F9(3)1781470311
3      (4)DA6645F6E22BF5F75974DC7EED5FCD6160D6B51E(5)100(6)113(7)
4 NONCE= (8)f4a8d8636e81d64d00000022ae2dcb1a
5 HASH = 00000000007BC41EE9F1021F56854A14CF2489DD793EC8C0CE8F360185AF6BA2
```

Code Listing 3: Block 2 – an Bob

```
1 IV = 00000000007BC41EE9F1021F56854A14CF2489DD793EC8C0CE8F360185AF6BA2
2 TX = (1)426BA(2)D92AB98BB95D0DA30A4C9B38CBEC5F2642AA73F9(3)1781470311
3      (4)6F90F101115140727C43CADEE0B9E17881403A63(5)7(6)111(7)
4 NONCE= (8)53da94499e68bd340000009f34dc1743
5 HASH = 00000000004BB8BD740A25D5B36C419B7668DBC34DC73352264D00C606466331
```

Code Listing 4: Block 3 – an Carol

Hinweis: In den TX-Strings ist der Zeilenumbruch vor (4) nur durch die Spaltenbreite bedingt; gehasht wird die ununterbrochene Zeichenkette.

Verifikation

Die SHA-256-Implementierung wird zunächst gegen den offiziellen Testvektor geprüft (reference.c, Selbsttest beim Start):

```
1 SHA256("abc") = BA7816BF8F01CFEA414140DE5DAE2223B00361A396177A9CB410FF61F20015AD
2 expected      = BA7816BF8F01CFEA414140DE5DAE2223B00361A396177A9CB410FF61F20015AD [OK]
```

Code Listing 5: Selbsttest gegen den bekannten SHA-256-Testvektor

Anschließend wird jeder Block-Hash mit einer dritten, völlig unabhängigen Implementierung – Pythons `hashlib` – nachgerechnet, z. B. für Block 1:

```
1 $ python3 -c "import hashlib; \
2   iv='0'*64; \
3   tx='(1)426B8(2)D92AB98BB95D0DA30A4C9B38CBEC5F2642AA73F9(3)1781470311' \
4     '(4)35318264C9A98FAF79965C270AC80C5606774DF1(5)42(6)112(7)'; \
5   nonce='(8)591b5f1056840691000000bc249430e6'; \
6   print(hashlib.sha256((iv+tx+nonce).encode()).hexdigest())"
7 0000000000161f40beaf0b2600ee1dd39e7b34f995702ad26f88e75a6053de94
```

Code Listing 6: Unabhängige Kontrolle mit Python `hashlib` (Block 1)

Alle drei Implementierungen (CUDA, C, Python) und CrypTool2 liefern für jeden Block denselben Hash; die zehn führenden Null-Nibbles sind reproduzierbar bestätigt.

Nachbau in CrypTool2

Der Workspace `Blockchain.cwm` bildet die Kette je Block mit Text Input → Concatenate → SHA-256 → Text Output nach:

1. Drei Text Input: IV (64 Hex), TX-String, (8)+NONCE.
2. Mit Concatenate in der Reihenfolge IV || TX || Nonce zusammenfügen.
3. In SHA-256 geben; das Text Output zeigt den Hash mit zehn führenden Nullen.
4. Der Output-Hash von Block n ist die IV-Eingabe von Block $n+1$ (Kette).

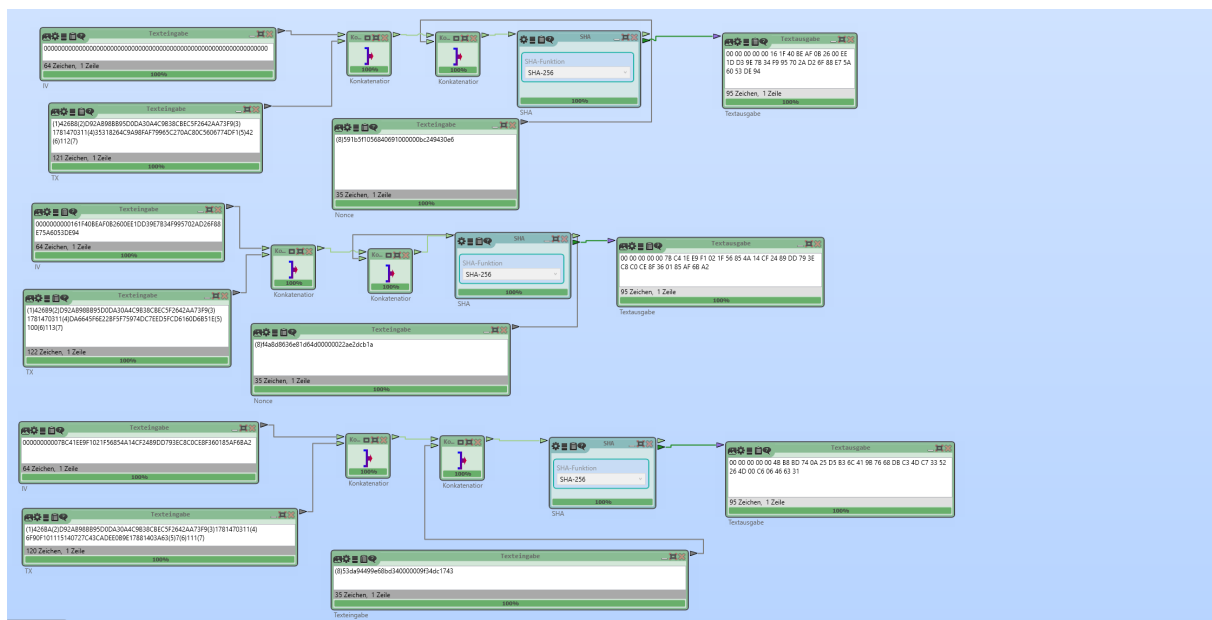


Abbildung 1: CrypTool2-Workspace (`Blockchain.cwm`): die drei verketteten Blöcke. Jeder Texteingabe-Strang (IV, TX, Nonce) wird per Konkatination zusammengesetzt und in SHA-256 gehasht. Die Textausgabe jedes Blocks zeigt den Hash mit zehn führenden Null-Nibbles (00 00 00 00 00 ...) und dient zugleich als IV-Eingabe des nächsten Blocks – die Verkettung ist damit sichtbar erfüllt.

Bewertung

- Alle drei Block-Hashes erfüllen die Anforderung mit $k = 10$ führenden Null-Nibbles; die Verkettung über den IV ist lückenlos.

- SHA-256 lässt sich nicht “austricksen” – der einzige Hebel ist roher Durchsatz. Der Midstate-Trick (konstante Blöcke vorhashen) und die GPU bringen k klar über das Musterbeispiel.
- Drei unabhängige SHA-256-Implementierungen (CUDA, C, Python) sowie CrypTool2 liefern identische Hashes – die Lösung ist reproduzierbar verifiziert.